

INGENIERÍA DE SOFTWARE EN CONTEXTO	3
INTRODUCCIÓN A LA INGENIERÍA DEL SOFTWARE. ¿QUÉ ES?	3
¿Qué es software?	3
¿Qué es ingeniería de software?	3
ESTADO ACTUAL Y ANTECEDENTES: LA CRISIS DEL SOFTWARE.....	3
¿Qué fue la "Crisis del Software"?.....	4
DISCIPLINAS QUE CONFORMAN LA INGENIERÍA DE SOFTWARE.....	5
Disciplinas Técnicas:	5
Disciplinas de Gestión:	5
Disciplinas de Soporte (o Transversales):	5
EJEMPLOS DE GRANDES PROYECTOS DE SOFTWARE FALLIDOS Y EXITOSOS.....	5
CICLOS DE VIDA (MODELOS DE PROCESO) Y SU INFLUENCIA EN LA ADMINISTRACIÓN DE PROYECTOS DE SOFTWARE.....	5
Relación entre Proceso, Proyecto y Ciclo de Vida	6
Clasificación de los Ciclos de Vida	6
Influencia del Ciclo de Vida en la Administración de Proyectos	7
PROCESOS DE DESARROLLO EMPÍRICOS VS. DEFINIDOS.	7
Procesos Definidos (Tradicionales / Dirigidos por Plan).....	7
Procesos Empíricos (Adaptativos / Ágiles).....	7
CICLOS DE VIDA (MODELOS DE PROCESO) Y PROCESOS DE DESARROLLO DE SOFTWARE	8
VENTAJAS Y DESVENTAJAS DE C/U DE LOS CICLOS DE VIDA.....	9
Ciclo de Vida Secuencial (Modelo en Cascada / Waterfall).....	9
Variante Secuencial: Modelo en V.....	9
Ciclo de Vida Iterativo e Incremental.....	10
Ciclo de Vida Recursivo / Orientado a Riesgos (Modelo en Espiral de Boehm)	10
Criterios según las Características del Producto	11
Criterios según las Necesidades y Restricciones del Proyecto	11
Criterios según el Cliente, el Equipo y la Cultura Organizacional.....	11
COMPONENTES DE UN PROYECTO DE SISTEMAS DE INFORMACIÓN.	12
Los Componentes Principales:.....	12
La Triple Restricción en la Gestión del Proyecto.....	13
Características de un proyecto.....	13
Roles en la Gestión Tradicional (Procesos Definidos).....	14
Plan de proyecto	14
NO HAY BALAS DE PLATA	15
Dificultades Esenciales vs. Dificultades Accidentales.....	15
Las 4 Propiedades Esenciales del Software	15
Evaluación de las "Falsas Balas de Plata"	15
Ataques Prometedores sobre la Esencia (Las Soluciones Reales).....	16
El Ecosistema del Desarrollo Exitoso	16
Proceso Orgánico (Eje de Ejecución y Aprendizaje).....	16
GESTIÓN LEAN-ÁGIL DE PRODUCTOS DE SOFTWARE	17
GESTIÓN DE PRODUCTO	17
El Propósito y la Arquitectura del Valor.....	17
Eliminación del Desperdicio (Waste) y el Enfoque Lean	17
El Ecosistema de los Mínimos y la Validación de Hipótesis.....	17
La Propuesta de Valor Única (UVP) y el Producto Mínimo Viable (MVP)	18
MMF (Minimum Marketable Feature / Característica Mínima Comercializable):.....	18
MVF (Minimum Viable Feature / Característica Mínima Viable):	18
MMP (Minimum Marketable Product)	18
MLP (Minimum Lovable Product / Producto Mínimo Adorable):.....	19

FILOSOFÍA ÁGIL Y MANIFIESTO ÁGIL.....	19
¿Qué es la Filosofía Ágil?.....	19
El Manifiesto Ágil (2001)	19
Los 12 Principios del Manifiesto Ágil	20
REQUERIMIENTOS ÁGILES E HISTORIAS DE USUARIO (USER STORIES).....	20
Filosofía de los Requerimientos Ágiles	21
Niveles de Abstracción y Tipos de Requerimientos	21
User Stories: Concepto y las 3 "C"	22
Formato de Redacción y Modelado de Roles	23
El Modelo INVEST (Calidad de una User Story).....	23
Spikes (Historias de Investigación).....	24
ESTIMACIONES.....	24
El Cono de Incertidumbre (Cone of Uncertainty).....	24
Variables Clave Para Estimar	24
Técnicas Tradicionales de Estimación	25
ESTIMACIONES ÁGILES	25
Filosofía: Separar Tamaño de Duración.....	25
Estimación Relativa	25
La Unidad Ágil: Story Points (Puntos de Historia).....	25
Planning Poker (Poker de Planificación)	26
Métrica de Progreso y Duración: Velocidad (Velocity)	27
SCRUM	27
Scrum: El Framework.....	27
Los Principios de Scrum.....	27
El Motor: Empirismo y el Ciclo OODA.....	28
Valores de Scrum	28
Valores y Cultura: El Sistema Operativo de la Confianza	28
La Unidad Central: Una Célula Profesional.....	29
Roles en la Guía Expandida	29
La Autogestión y la Inteligencia Artificial.....	31
Eventos de Scrum.....	31
Diferencias entre Output y Outcome (La Doble Definición de "Hecho")	33
Artefactos	33
Pensamiento Orientado a Producto (Product Thinking vs. Project Thinking).....	34
Capacidad del Equipo en un Sprint	35
Capacidad vs. Velocidad	35
Planificación de Release y Sprint	36
Métricas y Herramientas	37
GESTIÓN DEL SOFTWARE COMO PRODUCTO	38
SCM.....	38
Conceptos Fundamentales de SCM	39
Las 4 Actividades y Elementos Principales de SCM.....	40
Planificación de SCM.....	40
Evolución de SCM en Ambientes Ágiles y DevOps.....	41

Ingeniería de Software en Contexto

Introducción a la Ingeniería del Software. ¿Qué es?

La ingeniería de software busca apoyar el desarrollo de software profesional, en lugar de la programación individual. Incluye técnicas que apoyan la especificación, el diseño y la evolución del programa, ninguno de los cuales son normalmente relevantes para el desarrollo de software personal.

Muchos suponen que el software es tan sólo otra palabra para los programas de cómputo. No obstante, cuando se habla de ingeniería de software, esto no sólo se refiere a los programas en sí, sino también a toda la documentación asociada y los datos de configuración requeridos para hacer que estos programas operen de manera correcta. Un sistema de software desarrollado profesionalmente es usualmente más que un solo programa. El sistema por lo regular consta de un número de programas separados y archivos de configuración que se usan para instalar dichos programas. Puede incluir documentación del sistema, que describe la estructura del sistema; documentación del usuario, que explica cómo usar el sistema, y los sitios web para que los usuarios descarguen información reciente del producto.

¿Qué es software?

Programas de cómputo y documentación asociada. Los productos de software se desarrollan para un cliente en particular o para un mercado en general.

¿Qué es ingeniería de software?

La ingeniería de software es una disciplina de ingeniería que se interesa por todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación. En esta definición se presentan dos frases clave:

Disciplina de ingeniería: Los ingenieros hacen que las cosas funcionen. Aplican teorías, métodos y herramientas donde es adecuado. Sin embargo, los usan de manera selectiva y siempre tratan de encontrar soluciones a problemas, incluso cuando no hay teorías ni métodos aplicables. Los ingenieros también reconocen que deben trabajar ante restricciones organizacionales y financieras, de modo que buscan soluciones dentro de tales limitaciones.

Todos los aspectos de la producción del software: La ingeniería de software no sólo se interesa por los procesos técnicos del desarrollo de software, sino también incluye actividades como la administración del proyecto de software y el desarrollo de herramientas, así como métodos y teorías para apoyar la producción de software.

Estado Actual y Antecedentes: La Crisis del Software

Conferencia de la OTAN / NATO (1968): El término "Ingeniería de Software" fue propuesto oficialmente en la conferencia patrocinada por el Comité Científico de la OTAN en Garmisch, Alemania, con el fin de debatir los graves problemas presentes en el desarrollo de sistemas

Década de 1970: Publicación de *The Mythical Man-Month* por Frederick Brooks (1975) e introducción del análisis estructurado por Tom DeMarco (1978)

Década de 1980: Aparición de los primeros grandes errores públicos de software. Publicación del célebre ensayo "*No Silver Bullet*" de Frederick Brooks (1987) e introducción del desarrollo incremental y lenguajes de cuarta generación.

Décadas de 1990 en adelante: Expansión masiva de Internet, la Programación Orientada a Objetos y el surgimiento del movimiento ágil (Scrum, Extreme Programming) para afrontar la velocidad y el cambio constante.

¿Qué fue la "Crisis del Software"?

A finales de la década de 1960 se hizo evidente que los enfoques artesanales e individuales ("el programador solitario") no escalaban al construir sistemas grandes y complejos. Las principales manifestaciones de la crisis fueron:

- **Costos e imprecisión presupuestaria:** Los proyectos sobrepasaban enormemente los presupuestos iniciales.
- **Incumplimiento de plazos:** Entregas con severos retrasos y cronogramas irreales.
- **Baja calidad y desconfianza:** El software resultante era frágil, inestable y propenso a fallas críticas en producción.
- **Insatisfacción del usuario:** El producto final no satisfacía las necesidades reales del cliente o negocio.
- **Mantenibilidad deficiente:** Inexistencia o mala calidad de la documentación y código confuso dificultan extender o adaptar el sistema a futuras versiones.

En la actualidad, la industria del software es el motor central de la economía mundial y la sociedad dependiente de la tecnología. Sin embargo, los problemas de calidad y retraso persisten por algunos factores clave:

- **Demandas crecientes:** A medida que la ingeniería de software resuelve problemas pasados, las exigencias del mercado crecen a mayor velocidad (sistemas más complejos, distribuidos y necesidad de entregas ultrarrápidas).
- **Bajas expectativas y falta de profesionalismo:** Muchas empresas continúan desarrollando software de forma fortuita o sin aplicar metodologías, estándares de calidad ni prácticas ingenieriles.
- **Deterioro del software:** A diferencia del hardware (que se desgasta físicamente), el software **no se desgasta pero sí se deteriora** con los cambios continuos que introducen efectos colaterales no deseados si no existe una arquitectura sólida.

La principal enseñanza que deja la crisis del software es que saber programar NO es suficiente. Se requiere una disciplina formal de ingeniería, gestión de proyectos, procesos y comunicación efectiva con los stakeholders.

Disciplinas que conforman la Ingeniería de Software.

Disciplinas Técnicas:

- Requerimientos: Identificación y especificación de lo que el sistema debe hacer
- Análisis y Diseño: Definición de la arquitectura, componentes y estructura del sistema.
- Construcción: Codificación e implementación técnica de la solución.
- Prueba (Testing): Verificación y validación para encontrar defectos y asegurar el correcto funcionamiento.
- Despliegue: Puesta en producción e instalación del producto para los usuarios.

Disciplinas de Gestión:

- Planificación de Proyecto: Estimación de tiempos, costos, recursos y definición de hitos.
- Monitoreo y Control de Proyectos: Seguimiento del progreso y toma de acciones correctivas ante desvíos.

Disciplinas de Soporte (o Transversales):

- Gestión de Configuración de Software (SCM): Identificación, control de cambios y mantenimiento de la integridad del producto a lo largo de su ciclo de vida.
- Aseguramiento de Calidad (SQA): Auditoría y verificación de que los procesos y productos cumplen con los estándares establecidos.
- Métricas: Medición cuantitativa de procesos, proyectos y productos para guiar la toma de decisiones.

Ejemplos de Grandes Proyectos de Software Fallidos y Exitosos

Los fallos de software no solo generan pérdidas millonarias, sino que en sistemas críticos pueden costar vidas humanas o la pérdida de misiones espaciales:

Mariner I (1962): Sonda espacial de la NASA dirigida a Venus que se desvió de su trayectoria y debió ser destruida a los 293 segundos del despegue

Hartford Coliseum (1978): El techo de un estadio se derrumbó por el peso de la nieve pocas horas después de un evento masivo

Ciclos de Vida (Modelos de Proceso) y su Influencia en la Administración de Proyectos de Software

El **Ciclo de Vida** de un proyecto de software es una representación o abstracción de un proceso que define las **fases principales** del desarrollo y el **orden** en el que se deben ejecutar. No especifica en detalle los procedimientos individuales o las herramientas particulares, sino la estructura organizativa y la secuencia de las etapas.

Relación entre Proceso, Proyecto y Ciclo de Vida

- **Ciclo de Vida:** Es la estructura abstracta que guía la secuencia y el orden de las fases.
- **Proceso de Desarrollo:** Es la implementación concreta de un ciclo de vida, constituyendo el conjunto articulado de actividades y métodos necesarios para construir el software.
- **Proyecto:** Es la unidad de gestión y organización temporaria que ejecuta el proceso utilizando personas y recursos para obtener como resultado un producto de software.

Diferencia entre Ciclo de Vida del Proyecto y del Producto:

- **Ciclo de Vida del Proyecto:** Es temporario; se inicia cuando arranca la planificación y termina formalmente cuando se entrega el producto final o se alcanzan los objetivos del proyecto. Su cumplimiento se mide contra el **Plan de Proyecto**.
- **Ciclo de Vida del Producto:** Es de mayor extensión; abarca desde la concepción de la idea, la fase de operaciones, los procesos de mantenimiento y actualización, hasta el retiro definitivo del producto del mercado. Su cumplimiento se mide contra la **Especificación de Requerimientos**.

Clasificación de los Ciclos de Vida

Secuencial (dirigido por planes / prescriptivo):

- **Características:** Las fases (requerimientos, diseño, codificación, pruebas) se ejecutan una tras otra de forma estrictamente lineal sin retorno imprevisto entre etapas.
- **Ejemplos:** Modelo en Cascada (*Waterfall*) y Modelo en V.
- **Uso apropiado:** Proyectos con requerimientos estables, muy bien comprendidos y con baja probabilidad de cambios

Iterativo / Incremental (empírico / ágil):

- **Características:** Entrelaza continuamente la especificación, el desarrollo y la validación en múltiples ciclos o versiones intermedias (incrementos).
- **Ejemplos:** Desarrollo Incremental, Scrum, Proceso Unificado.
- **Uso apropiado:** Proyectos complejos con alta incertidumbre, requerimientos cambiantes y necesidad de entregas tempranas de valor al cliente.

Recurso / Orientado a Riesgos:

- **Características:** Diseñado específicamente para evaluar y mitigar riesgos en cada iteración antes de avanzar a la construcción del producto.
- **Ejemplos:** Modelo en Espiral de Boehm.
- **Uso apropiado:** Proyectos de gran escala, altamente complejos o de alto riesgo técnico.

Influencia del Ciclo de Vida en la Administración de Proyectos

La selección del ciclo de vida condiciona directamente la manera en que el Project Manager o el equipo administra y controla el proyecto. Por ejemplo, si se realiza un análisis de riesgo y se determina que los requerimientos pueden variar mucho a lo largo del desarrollo, se debería elegir un ciclo de vida en el cual el cambio sea permisible, como los iterativos. En este caso, la administración y la planificación del proyecto se ve afectada en la elección del ciclo de vida. Además, durante el desarrollo del proyecto, la gestión de este se realiza en consecuencia de esta elección, ya que no es lo mismo gestionar un proyecto que se desarrolla de manera iterativa, que uno que se desarrolla de manera secuencial o en cascada.

Procesos de Desarrollo Empíricos vs. Definidos.

Un proceso es una secuencia de pasos ejecutados para un propósito dado (IEEE), mientras que un proceso de software se define como un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados. Las actividades del proceso de desarrollo son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba. El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida.

Procesos Definidos (Tradicionales / Dirigidos por Plan)

- **Inspiración:** Se basan en una concepción derivada del modelo industrial y las líneas de producción en masa.
- **Premisa fundamental:** Asumen que, al introducir un conjunto determinado de entradas (*inputs*) y ejecutar una secuencia bien definida de actividades intermedias, la salida (*output*) será siempre predecible y repetible de forma indefinida.
- **Mecanismo de administración y control:** La gestión y el control provienen directamente de la **predictibilidad** del proceso fijado por adelantado.
- **Inaplicabilidad en el software:** El software posee una complejidad esencial e intangibilidad que impiden simplificar el desarrollo a un flujo estático con resultados predecibles a partir de entradas fijas.
- **Ejemplos:** El Proceso Unificado (PUD / RUP) o los modelos secuenciales clásicos en cascada.

Procesos Empíricos (Adaptativos / Ágiles)

- **Inspiración y concepto:** Proviene del **empirismo**, corriente que postula que el conocimiento proviene de la experiencia práctica y de tomar decisiones en función de lo observado en el mundo real.
- **Premisa fundamental:** Asumen que el desarrollo de software involucra variables cambiantes en entornos complejos y creativos, por lo que al repetir el proceso se pueden obtener resultados diferentes.
- **Mecanismo de administración y control:** No se apoyan en un plan rígido preestablecido, sino en la **inspección frecuente y la adaptación continua**.
- **Centro de gravedad:** Colocan el foco en las **personas**, quienes son la clave del empirismo al capitalizar el aprendizaje técnico y de negocio en cada ciclo
- **Ejemplos:** Frameworks ágiles como **Scrum**

Para estructurar el aprendizaje continuo y la evolución del producto, los procesos empíricos aplican un ciclo de retroalimentación de 5 pasos:

- **Asumir:** Se plantea una hipótesis o requisito inicial.
- **Construir:** Se elabora un incremento ejecutable o Producto Mínimo Viable (MVP).
- **Retroalimentar:** Se obtiene información real a partir de la inspección e interacción con el cliente o usuario.
- **Revisar:** Se analizan los datos recolectados y se contrasta el resultado frente a la hipótesis original en una retrospectiva.
- **Adaptar:** Se realizan ajustes en el producto y en el proceso para guiar la siguiente iteración de construcción

Los 3 pilares del empirismo que sustentan el control de procesos empíricos (y sirven de pilar fundamental para frameworks ágiles como Scrum) son:

- **Transparencia:** El proceso y el trabajo en desarrollo deben ser totalmente visibles e inteligibles tanto para quienes realizan el trabajo como para quienes lo reciben. Permite que las decisiones críticas se tomen basándose en la realidad observada y en el estado real de los artefactos.
- **Inspección:** Los artefactos y el progreso hacia los objetivos acordados se deben examinar con frecuencia y diligencia para detectar oportunamente variaciones o problemas indeseables.
- **Adaptación:** Si al inspeccionar se observa que algún aspecto del proceso o del producto se desvía fuera de los límites aceptables, el trabajo o el proceso debe ajustarse lo antes posible para minimizar un impacto mayor

Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software

¿Qué es un Ciclo de Vida de Software?

Define la hoja de ruta teórica sobre cómo progresa un producto o proyecto a lo largo del tiempo. No detalla qué procedimientos específicos usar ni quién debe realizarlos, sino únicamente la secuencia organizativa de las fases.

¿Qué es un Proceso de Desarrollo de Software?

Es el conjunto estructurado de **actividades, métodos, prácticas y transformaciones** que utiliza el equipo de trabajo para desarrollar o mantener el software y sus productos asociados

Ventajas y desventajas de c/u de los ciclos de vida.

Ciclo de Vida Secuencial (Modelo en Cascada / Waterfall)

En este modelo, las actividades se ejecutan una tras otra de forma lineal y cada fase debe completarse y documentarse formalmente antes de iniciar la siguiente.

Ventajas:

- **Simplicidad y claridad:** Es una estructura fácil de entender, planificar y administrar.
- **Alta visibilidad del avance:** Produce documentación formal e hitos bien definidos al final de cada etapa, lo que facilita el control directivo respecto a un plan inicial.
- **Ideal para requerimientos estables:** Funciona de manera óptima cuando las necesidades del cliente están perfectamente comprendidas desde el inicio y es improbable que cambien.

Desventajas:

- **Inflexibilidad ante cambios:** Es sumamente rígido; incorporar cambios a requerimientos en etapas avanzadas genera gran confusión y altos costos por rehacer trabajo.
- **Retraso en la entrega funcional:** El cliente no dispone de una versión ejecutable del software hasta las etapas finales del proyecto.
- **Riesgo de fallas catastróficas:** Si se descubre un error de diseño o especificación en la fase de pruebas o despliegue, la corrección es desastrosa y muy costosa.
- **Estados de bloqueo:** Se generan tiempos muertos en el equipo cuando ciertas tareas dependen de la aprobación formal de la fase anterior.

Variante Secuencial: Modelo en V

Relaciona cada fase del desarrollo inicial (izquierda de la V) con su correspondiente nivel de prueba y validación (derecha de la V).

Ventajas:

- **Aseguramiento explícito de la calidad:** Hace muy visible la correspondencia entre los modelos de diseño/requerimientos y las acciones de verificación.

Desventajas:

- Mantiene la misma inflexibilidad estructural que la Cascada frente a requerimientos inestables o cambiantes.

Ciclo de Vida Iterativo e Incremental

Divide el desarrollo en una serie de versiones (incrementos) y ciclos cortos de especificación, desarrollo y validación entrelazados, constituyendo la base de los enfoques empíricos y ágiles.

Ventajas:

- **Tolerancia y adaptación al cambio:** Reduce significativamente el costo de adaptar requerimientos cambiantes respecto al modelo en cascada.
- **Retroalimentación continua del cliente:** El usuario evalúa software ejecutable desde los primeros incrementos, facilitando la validación del producto real.
- **Entrega temprana de valor:** Las funcionalidades más críticas se entregan antes, permitiendo al cliente obtener valor de negocio en etapas tempranas del proyecto.

Desventajas:

- **Baja visibilidad burocrática:** Al priorizar software funcional sobre documentos extensos, puede resultar difícil medir el avance para administradores tradicionales.
- **Degradación de la arquitectura:** Si no se realiza refactorización continua, la estructura del código tiende a corromperse a medida que se agregan más incrementos.
- **Dificultad en proyectos grandes o con contratos rígidos:** Complicado de aplicar si la organización cliente exige un contrato cerrado con la especificación completa por adelantado (*BRUF*)

Ciclo de Vida Recursivo / Orientado a Riesgos (Modelo en Espiral de Boehm)

Representa el proceso como una espiral donde cada ciclo o vuelta abarca el análisis de alternativas, evaluación explícita de riesgos, desarrollo y evaluación.

Ventajas:

- **Gestión explícita del riesgo:** Identifica y resuelve activamente incertidumbres técnicas y de negocio en cada iteración antes de comprometer recursos masivos.
- **Combina lo mejor de dos mundos:** Une la flexibilidad de los prototipos con el control estructurado de los modelos prescriptivos.
- **Abarca todo el ciclo de vida del producto:** Se puede adaptar tanto para la concepción inicial como para posteriores actualizaciones y mantenimiento del software.

Desventajas:

- **Requiere alta experiencia:** Exige profesionales altamente capacitados en la identificación y evaluación precisa de riesgos.
- **Dificultad contractual:** Resulta complejo convencer a clientes o gerencias acostumbrados a esquemas fijos de que el proceso evolutivo es controlable.
- **Incertidumbre en la planificación:** Dificulta la estimación inicial del número total de circuitos necesarios para terminar el producto

Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.

No existe un modelo de ciclo de vida que sea universalmente el más "rápido", ya que el modelo más efectivo depende totalmente del contexto en el que se aplique. Un ciclo de vida que funciona de manera óptima en una situación puede fallar drásticamente si se aplica de forma incorrecta en otra, por lo que la velocidad de desarrollo está siempre condicionada a las particularidades del proyecto.

Criterios según las Características del Producto

Este tipo de criterio se enfoca en las propiedades intrínsecas y técnicas de la solución de software que se busca construir. Considera de forma primordial la **estabilidad, claridad o volatilidad de los requerimientos**, distinguiendo entre sistemas con funcionalidades bien comprendidas desde el inicio y aquellos expuestos a cambios continuos. Asimismo, evalúa la **escala, complejidad y nivel de criticidad técnica o de seguridad del producto**, donde posibles fallos en producción acarrearán consecuencias graves y exigen una evaluación explícita de riesgos mediante prototipado o análisis iterativo. De este modo, los productos estables y con normas estrictas se orientan hacia modelos secuenciales o formales, mientras que los productos complejos, modulares o cambiantes requieren ciclos de vida iterativos e incrementales.

Criterios según las Necesidades y Restricciones del Proyecto

Esta categoría abarca las variables operativas, temporales y financieras que condicionan la planificación y dirección de la iniciativa. Evalúa de manera prioritaria la **urgencia en el tiempo de salida al mercado (time-to-market)** y la necesidad de realizar entregas funcionales tempranas para aportar valor constante al negocio. También contempla el **esquema presupuestario y contractual** (contratos cerrados de alcance fijo versus esquemas con tiempo/costos fijos y alcance variable) y el **marco de gestión seleccionado**. Mientras que la gestión tradicional con procesos definidos otorga libertad teórica para aplicar cualquier ciclo de vida, la gestión ágil empírica impone la restricción metodológica de utilizar únicamente ciclos de vida iterativos e incrementales.

Criterios según el Cliente, el Equipo y la Cultura Organizacional

Este tipo de criterio analiza los factores humanos, socioculturales y de gobernanza presentes en el entorno del desarrollo. Examina el **grado de disponibilidad y participación del cliente o usuario final**, diferenciando entre quienes colaboran continuamente en cada iteración y quienes solo intervienen en la especificación inicial y la validación final. Además, toma en cuenta la **madurez, tamaño y experiencia del equipo de desarrollo**, junto con la **cultura organizativa** y la presencia de entes reguladores externos que exijan documentación exhaustiva. Organizaciones burocráticas o normadas favorecen ciclos prescriptivos tradicionales, mientras que equipos reducidos e integrados operan con mayor eficacia bajo ciclos iterativos e incrementales.

Componentes de un Proyecto de Sistemas de Información.

Un proyecto de desarrollo de software es la unidad organizativa y de gestión temporaria que administra recursos, tiempos y personas para construir un producto.

Un proyecto es llevado a cabo por personas que aplican herramientas para automatizar un proceso y obtener como resultado un producto de software.



Los Componentes Principales:

- **Personas (Personal / Capital Humano):** Constituyen el principal capital y recurso consumido en el desarrollo de software. Incluyen a los individuos dotados de habilidades, entrenamiento y motivación para realizar las actividades. Abarca diversos roles como el *Project Manager*, desarrolladores, analistas, testers, clientes y usuarios finales.
- **Proceso:** Es el conjunto estructurado de actividades, métodos, prácticas y transformaciones que utiliza el equipo para desarrollar o mantener el software y sus productos asociados. Funciona como una plantilla o definición teórica abstracta que expresa qué se debe hacer, la cual se instancia y se adapta a las necesidades específicas del proyecto.
- **Herramientas y Equipos:** Comprende la infraestructura tecnológica, los entornos de desarrollo (IDE, herramientas CASE), hardware, repositorios y sistemas de soporte que automatizan las tareas del proceso e incrementan la eficiencia operativa.
- **Producto:** Es el resultado final que se obtiene al concluir el proyecto. Incluye los programas ejecutables, la documentación técnica y de usuario, y los datos que aportan valor al cliente.
- **Proyecto:** Es la unidad de gestión de duración limitada (con inicio y fin definidos) orientada a alcanzar un objetivo único y claro. El proyecto coordina a las personas, administra el presupuesto y adapta el proceso para obtener el producto esperado.

La Triple Restricción en la Gestión del Proyecto

Para asegurar la **calidad** del resultado (la cual no es negociable), la dirección del proyecto debe equilibrar tres variables que compiten entre sí:

1. **Alcance:** El objetivo y las funcionalidades a construir.
2. **Tiempo:** El plazo o cronograma comprometido para la entrega.
3. **Costo:** El presupuesto y los recursos asignados.

Diferencia entre Alcance del Producto y Alcance del Proyecto:

- **Alcance del Producto:** Son todas las características y funciones del producto de software. Su cumplimiento **se mide contra la Especificación de Requerimientos**.
- **Alcance del Proyecto:** Es todo el trabajo (y solo el trabajo) necesario para entregar el producto con sus características especificadas. Su cumplimiento **se mide contra el Plan de Proyecto**.



Características de un proyecto

- **Únicos:** El resultado de un proyecto será único e irrepetible para cualquier otro proyecto, por más parecido o igual sean sus elementos componentes.
- **Orientados a un objetivo:** (esto me permite que los proyectos sean únicos, para ello el objetivo tiene que ser claro, no ambiguo y alcanzable), tengo que poder definir hacia dónde voy y poder establecer/medir cuando alcancé ese objetivo para poder dar por finalizado el proyecto. Los proyectos tienen un objetivo único, es decir que cada resultado de un proyecto es distinto del resultado de otro proyecto.
- **Duración limitada de tiempo:** Esto implica que un proyecto tiene un principio y un fin, liberando los recursos y personas. Cuando se alcanzan los objetivos del proyecto, este llega a su fin. Un ejemplo claro de lo que no es un proyecto es una línea de producción, ya que esta se ejecuta indefinidamente en el tiempo.
- **Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos:** Se definen las tareas, sus dependencias, los recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

Roles en la Gestión Tradicional (Procesos Definidos)



Líder de Proyecto / Project Manager (PM): Es el responsable de las tareas de gestión operativa, planificación, estimación y seguimiento del proyecto. Administra los recursos, organiza las asignaciones y busca balancear la **triple restricción** (alcance, tiempo y costo) para garantizar la calidad del producto.

Equipo de Proyecto: Grupo multidisciplinario de personas (analistas, arquitectos/diseñadores, programadores, testers) que ejecutan las actividades técnicas requeridas para transformar los requerimientos en software funcional.

Participantes y Stakeholders:

- **Cliente / Patrocinador:** Persona o entidad que solicita la construcción del software, define los objetivos del negocio y financia el proyecto.
- **Usuarios Finales:** Personas que interactuarán directamente con el sistema en su día a día y definen los detalles operativos.
- **Niveles Altos de Administración / Gerentes Ejecutivos:** Definen la estrategia del negocio y asignan los recursos generales a la iniciativa

Plan de proyecto

En la Ingeniería de Software, el Plan de Proyecto es el instrumento fundamental de gestión que actúa como una "hoja de ruta" para guiar al equipo de trabajo a lo largo del desarrollo de un sistema.

Componentes e Implicaciones de la Planificación:

- **Definición del Alcance:** Es todo el trabajo (y solo el trabajo) necesario para entregar el producto. Su cumplimiento se mide contra el Plan de Proyecto.
- **Definición de Proceso y Ciclo de Vida:** Se selecciona la estructura metodológica (secuencial, iterativa o recursiva) que organizará la secuencia de las fases.
- **Estimaciones de Software:** Se calculan bajo cierto nivel de incertidumbre las variables de Tamaño, Esfuerzo (horas/meses hombre), Calendario, Costo y Recursos Críticos.
- **Gestión de Riesgos:** Identificación, análisis, planificación de mitigación y seguimiento de aquellos eventos potenciales que podrían comprometer el éxito de la iniciativa.
- **Definición de Métricas, Monitoreo y Control:** Establecimiento de indicadores de seguimiento para comparar lo planificado frente a lo real en la ejecución y corregir desviaciones a tiempo.

No hay balas de plata

No existe ningún desarrollo tecnológico ni técnica de gestión que por sí sola prometa una mejora de un orden de magnitud (10x) en productividad, fiabilidad o simplicidad en la ingeniería de software dentro de una década.

Dificultades Esenciales vs. Dificultades Accidentales

Dificultades Esenciales: Aquellas inherentes a la propia naturaleza de la entidad conceptual del software. Son inevitables e incurables.

Dificultades Accidentales: Aquellas asociadas a la producción o a las herramientas utilizadas en una época determinada (ej. sintaxis engorrosa, restricciones de memoria, tiempos de compilación lentos o procesamiento por lotes), pero que no son inherentes al software en sí.

Las 4 Propiedades Esenciales del Software

Complejidad: Las entidades de software son más complejas que casi cualquier otra construcción humana porque **no hay dos partes iguales**. A diferencia de autos o edificios con partes repetidas, si dos elementos de software se parecen, se unifican en una función o subrutina. Si se elimina la complejidad en un modelo, se destruye su esencia.

Conformidad: El software no se diseña en un vacío con leyes físicas unificadas; debe adaptarse e interconectarse con interfaces humanas, regulaciones e instituciones arbitrarias e ilógicas.

Variabilidad / Inconstancia (Mutability): El software sufre una presión constante para ser modificado. Esto ocurre porque vive en entornos cambiantes y porque se lo percibe como algo infinitamente maleable en comparación con el hardware.

Invisibilidad: El software es inherentemente **invisualizable**. No posee una representación geométrica o espacial directa, lo que dificulta que la mente humana reflexione sobre su diseño o lo comunique a otros miembros del equipo.

Evaluación de las "Falsas Balas de Plata"

- **Ada y lenguajes modernos de alto nivel:** Facilitan la modularidad y los tipos abstractos, pero son solo evoluciones; el gran salto ya ocurrió al pasar del ensamblador al lenguaje compilado.
- **Programación Orientada a Objetos (POO):** Ayuda a reutilizar código y eliminar redundancias sintácticas, pero no altera la complejidad conceptual del diseño del sistema.
- **Inteligencia Artificial y Sistemas Expertos:** Lo difícil al construir software es **decidir qué se quiere decir** (la especificación conceptual) y no la forma de expresarlo. Los sistemas expertos solo aplican bien a dominios restringidos con reglas explícitas.
- **Verificación Formal de Programas:** Probar matemáticamente la corrección de un programa requiere tanto trabajo que solo resulta viable para núcleos críticos de seguridad.
- **Entornos e IDEs de Desarrollo:** Evitan errores sintácticos o semánticos simples, pero no resuelven la creación del modelo mental del software.

Ataques Prometedores sobre la Esencia (Las Soluciones Reales)

Aunque no existe una bala de plata, Brooks propone un camino disciplinado de mejoras continuas para abordar la **complejidad conceptual**:

Comprar vs. Construir (Software comercial / COTS): La solución más radical para evitar construir software es no hacerlo. Adquirir paquetes genéricos o herramientas de uso masivo (como hojas de cálculo o bases de datos) permite repartir el costo de desarrollo entre miles de usuarios.

Prototipado Rápido y Especificación Iterativa: Es imposible que un cliente especifique de entrada todos los requerimientos precisos sin probar el sistema. El prototipado permite validar hipótesis y descubrir iterativamente lo que el cliente realmente necesita.

Desarrollo Incremental ("Hacer crecer" el software): Se debe abandonar la metáfora de la construcción física e imitar a los organismos vivos. El software debe **crecer de forma orgánica**: lograr primero un esqueleto mínimo ejecutable e ir rellenando gradualmente las funcionalidades en iteraciones sucesivas.

Grandes Diseñadores: Las diferencias de rendimiento entre un programador promedio y uno excepcional son enormes. La estrategia principal de las organizaciones debe ser identificar, cultivar y retener grandes diseñadores creativos capaces de preservar la **integridad conceptual** del sistema.

El Ecosistema del Desarrollo Exitoso

Integridad Conceptual (Eje Arquitectónico)

El principio de la Mente Maestra: La integridad conceptual es la consideración más importante en el diseño de un sistema. Para que un software sea coherente, elegante y fácil de mantener, la arquitectura general debe ser guiada por una visión unificada (un arquitecto jefe o diseño centralizado)

Estructuras de datos limpias y ocultación de información: Se basa en modularizar el sistema aislando los detalles de implementación complejos detrás de interfaces estables. Esto reduce el acoplamiento y evita que las modificaciones en un componente propaguen fallos al resto del sistema

Personas Empoderadas (Eje Sociológico y Cultural)

Equipos Quirúrgicos Pequeños: En lugar de agregar ejércitos de programadores que ralentizan el proyecto debido a la sobrecarga de comunicación (Ley de Brooks), se conforman equipos pequeños, multidisciplinarios y altamente capacitados articulados alrededor del diseñador principal.

Proceso Orgánico (Eje de Ejecución y Aprendizaje)

"Hacer crecer" en lugar de "construir": Brooks plantea abandonar la metáfora rígida de la construcción civil y tratar al software como un organismo vivo que crece de forma orgánica. Se parte de un esqueleto mínimo funcional que se ejecuta desde el primer momento y se va enriqueciendo por capas.

Gestión Lean-Ágil de Productos de Software

Gestión de producto

Aborda cómo concebir, validar y evolucionar productos de software maximizando el valor entregado y eliminando el desperdicio.

El Propósito y la Arquitectura del Valor

Visión de Producto: Crear un producto de software responde a una visión de largo plazo para resolver problemas reales y transformar el contexto del usuario.

Alineación Estratégica: Conecta la Visión del Producto (largo plazo) con el Roadmap, el Product Goal (mediano plazo) y el Sprint Goal (corto plazo).

Eliminación del Desperdicio (Waste) y el Enfoque Lean

La **eliminación del desperdicio (waste)** y el **enfoque Lean** constituyen uno de los pilares filosóficos y estratégicos más importantes de la ingeniería de software moderna y la gestión de productos. No se trata simplemente de "trabajar más rápido", sino de descubrir e implementar la solución correcta maximizando el valor de negocio y reduciendo al mínimo las tareas que no aportan ningún beneficio real.

El enfoque Lean propone transformar la perspectiva tradicional: la productividad de un equipo no se mide en la cantidad de líneas de código producidas ni en la cantidad de documentación generada, sino en el impacto real y en el valor entregado al negocio.

El Ecosistema de los Mínimos y la Validación de Hipótesis

Muchas organizaciones aplazan el lanzamiento de sus productos esperando alcanzar una versión "perfecta" o "completa" por temor a mostrar cero clientes o cero ingresos. Esta postergación genera un enorme **Costo de Retraso** (*Cost of Delay*), perdiendo información esencial del mercado y acumulando trabajo desperdiciado.

Para combatir la parálisis por análisis, el desarrollo se organiza como un experimento científico continuo:

1. **Idea / Hipótesis:** Se formula una suposición sobre la necesidad del usuario.
2. **Construir (Producto Mínimo):** Se elabora la versión mínima indispensable para probar esa idea.
3. **Medir / Experimentar (Datos):** Se expone el experimento a usuarios reales para obtener métricas objetivas (retención, uso, interacción) y no simples opiniones.
4. **Aprender (Conocimiento Validado):** Se analiza la información recolectada para decidir si continuar, ajustar o realizar un **Pivote**.

La Propuesta de Valor Única (UVP) y el Producto Mínimo Viable (MVP)

UVP (User Value Proposition / Propuesta de Valor Única): Es la hipótesis central sobre el valor diferenciador que resolverá el problema o necesidad del usuario.

MVP (Minimum Viable Product / Producto Mínimo Viable): Es la versión de un nuevo producto desarrollada con el **menor esfuerzo posible** para obtener **aprendizaje validado** sobre la UVP en un subconjunto de clientes potenciales (*early adopters*).

- **Objetivo real: No es vender ni generar retornos financieros**, sino comprobar si la hipótesis del producto funciona o si debe modificarse.
- **Anatomía de un MVP efectivo (Minimalismo vs. Viabilidad):** Un error común es recortar excesivamente el desarrollo hasta entregar un producto fragmentado o inservible. Un MVP debe ser **mínimo pero cohesivo y viable**, abarcando la porción vertical necesaria de funcionalidad, fiabilidad, usabilidad y diseño

Al exponer el MVP ante los usuarios, se pueden obtener dos tipos de señales:

- **Señal Dispersa (Falta de mercado):** Si cada cliente potencial solicita características totalmente distintas ("es genial, pero necesito X"), indica que la visión aún no está clara y no existe un mercado definido para esa solución.
- **Señal Clara (El Pivote):** Si la retroalimentación de los usuarios apunta de manera consistente hacia la misma variación o necesidad no contemplada, se realiza un **pivote**, redefiniendo la propuesta y construyendo un **MVP2** basado en el aprendizaje adquirido

MMF (Minimum Marketable Feature / Característica Mínima Comercializable):

- Es la pieza más pequeña de funcionalidad que se puede liberar aportando valor comercial inmediato y retorno de inversión tanto para la organización como para los usuarios.
- Mientras el MVP busca aprender, el MMF busca ganar y escalar sobre certezas ya comprobadas.

MVF (Minimum Viable Feature / Característica Mínima Viable):

- Es una función "pionera" de alta incertidumbre dentro de un producto existente. Funciona como una "mini versión de un MVP" para explorar una nueva área de valor sin comprometer la estabilidad genera

MMP (Minimum Marketable Product)

- Es el primer release o entrega comercial dirigida a early adopters que combina los MMFs y MVFs esenciales. Representa el incremento más pequeño que satisface las necesidades del mercado inicial. Los lanzamientos posteriores serán **MMR2, MMR3... MMR N**.

MLP (Minimum Lovable Product / Producto Mínimo Adorable):

- **Más allá de la viabilidad:** En mercados maduros dominados por competidores consolidados, un producto que solo es "funcional" resulta insuficiente.
- El MLP busca **cruzar la grieta de la experiencia**: no solo resuelve el problema, sino que reduce la fricción, aporta deleite y crea una **conexión emocional temprana** para asegurar la lealtad y retención del usuario (ejemplos: la fluidez de WhatsApp, el descubrimiento personalizado de Spotify).

Filosofía Ágil y Manifiesto Ágil

La Filosofía Ágil (o Agilismo) surge como una respuesta cultural y técnica a los problemas de la gestión tradicional prescriptiva, buscando un equilibrio entre la falta de estructura y la burocracia excesiva

¿Qué es la Filosofía Ágil?

Ideología, no metodología: El agilismo **no es un proceso rígido ni una metodología prescriptiva**, sino una ideología y un marco mental (mindset) guiado por principios que orientan el desarrollo de productos de software.

El compromiso útil: Plantea un punto medio entre "nada de proceso" (caos e indisciplina) y "demasiado proceso" (burocracia paralizante). Ser ágil no significa trabajar de forma indisciplinada, sino aplicar prácticas ingenieriles con economía de acción.

Base Empírica y Adaptativa: Se sustenta sobre **procesos empíricos** que reconocen que la incertidumbre y los cambios en el software son inevitables. En lugar de pretender predecir todo al inicio, se apoya en ciclos cortos de retroalimentación (inspección y adaptación constante).

Orientado a la gente: Privilegia la capacidad de las personas y la colaboración de los equipos por encima de la rigidez de los procesos o el control jerárquico.

El Manifiesto Ágil (2001)

Este manifiesto declara **4 Valores Ágiles** fundamentales donde, aunque se reconoce el valor de los elementos de la derecha, **se priorizan los de la izquierda**

1. **Individuos e interacciones por sobre procesos y herramientas:** Enfatiza los vínculos humanos, la comunicación fluida y la colaboración directa frente a la imposición de herramientas o procedimientos
2. **Software funcionando por sobre documentación exhaustiva:** Prioriza la entrega de incrementos funcionales de alta calidad. La documentación no se elimina, sino que se genera "*Just In Time*" (cuando hace falta) y de forma Lean, evitando la burocracia de los modelos tradicionales.
3. **Colaboración con el cliente por sobre negociación contractual:** Promueve el trabajo en equipo continuo con el cliente (Product Owner) para descubrir sus necesidades reales, superando la inflexibilidad de los contratos cerrados
4. **Responder al cambio por sobre seguir un plan:** Asume que los requerimientos cambiarán a lo largo del desarrollo; por ello, valora la flexibilidad para ajustar el rumbo por encima del apego ciego a un plan inicial

Los 12 Principios del Manifiesto Ágil

Los valores se operacionalizan mediante 12 principios que guían la práctica diaria del equipo:

1. **Satisfacción del cliente:** La máxima prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor real.
2. **Aceptación del cambio:** Los cambios en los requerimientos son bienvenidos, incluso en etapas tardías, utilizándolos como ventaja competitiva para el cliente.
3. **Entregas frecuentes:** Entregar software funcional de manera periódica, en intervalos de pocas semanas a un par de meses (priorizando los plazos más cortos).
4. **Trabajo en equipo continuo:** La gente de negocio y los desarrolladores deben trabajar juntos diariamente durante todo el proyecto.
5. **Individuos motivados:** Los proyectos se construyen en torno a personas motivadas, brindándoles el entorno, el apoyo y la confianza necesarios.
6. **Comunicación cara a cara:** La conversación directa y cara a cara es el método más eficiente y efectivo para transmitir información.
7. **Software funcionando como medida de avance:** El software funcional e integrado es el indicador principal de progreso del proyecto.
8. **Desarrollo sostenible:** Promueve mantener un ritmo de trabajo constante e indefinido por parte de patrocinadores, desarrolladores y usuarios.
9. **Excelencia técnica:** La atención continua a la calidad técnica y al buen diseño mejora la agilidad y previene la degradación del sistema.
10. **Simplicidad:** Definida como *"el arte de maximizar la cantidad de trabajo no realizado"*, evitando agregar funcionalidades innecesarias que el cliente no ha pedido.
11. **Equipos auto-organizados:** Las mejores arquitecturas, requerimientos y diseños emergen de equipos con autonomía operativa.
12. **Mejora continua:** El equipo reflexiona a intervalos regulares sobre cómo volverse más efectivo para ajustar y perfeccionar su comportamiento.

Requerimientos Ágiles e Historias de Usuario (User Stories)

En el enfoque ágil, la gestión de requerimientos cambia de forma radical respecto a la ingeniería de software tradicional. En lugar de intentar predecir y documentar exhaustivamente todas las funcionalidades al inicio del proyecto en la práctica conocida como **BRUF (Big Requirements Up Front)**, los requerimientos ágiles se conciben como un proceso continuo de descubrimiento y maduración en colaboración directa con el cliente.

Filosofía de los Requerimientos Ágiles

1. **Inversión respecto al enfoque tradicional:** En la gestión tradicional (dirigida por un plan), los requerimientos se mantienen fijos mientras que el tiempo y los costos son variables. En la **gestión ágil**, el **Tiempo** (sprints) y los **Recursos/Costos** (equipo estable) se mantienen **fijos**, mientras que el **Alcance** es **variable** para responder al cambio y priorizar el valor de negocio.
2. **Foco en el Outcome sobre el Output:** Lo verdaderamente crucial no es entregar una salida técnica o un requerimiento (Output), sino entregar una solución que genere un impacto y resultado de valor real para el negocio (Outcome).
3. **Análisis Just In Time (JIT):** Se difiere el análisis detallado de los requerimientos para realizarlo "tan tarde como sea posible", es decir, justo antes de que comience el trabajo de implementación de esa historia. Esto evita el desperdicio de tiempo analizando funcionalidades que luego cambian o se descartan.
4. **Aceptar el cambio como ventaja competitiva:** Como el usuario suele descubrir lo que realmente necesita al interactuar con el software funcional ("El usuario dice lo que quiere cuando recibe lo que pidió"), los requerimientos se gestionan en una lista dinámica y re-priorizable.

Niveles de Abstracción y Tipos de Requerimientos

Los requerimientos pueden clasificarse según su dominio y nivel de abstracción:

- **Dominio del Problema:** Comprende los **Requerimientos de Negocio** (objetivos de alto nivel de la organización) y los **Requerimientos de Usuario** (necesidades del usuario final).
- **Dominio de la Solución:** Incluye los **Requerimientos Funcionales** (comportamiento del sistema), **No Funcionales** (restricciones de rendimiento, seguridad o calidad) y de **Implementación** (infraestructura o tecnología).
- **Jerarquía de Abstracción:** Los requerimientos en el *Product Backlog* evolucionan en nivel de detalle:
 - **Temas (Themes):** Agrupaciones de requerimientos de muy alto nivel a escala organizacional.
 - **Épicas (Epics):** Historias de gran tamaño que abarcan múltiples iteraciones y deben descomponerse más adelante.
 - **Historias de Usuario (User Stories):** Unidades funcionales mínimas desglosadas que cumplen los criterios para ser implementadas dentro de un único *Sprint*.

En este contexto, algunos conceptos relacionados con los principios ágiles que aplicamos son:

- Los cambios van a existir todo el tiempo de forma constante: Sabemos que hay cambio, por lo que se busca gestionar los requerimientos orientados a poder aceptar el cambio. Si el Product Owner, el cliente o los stakeholders relacionados con el producto no plantean cambios es porque el producto no está siendo utilizado.
- Debemos tener una mirada de todos los que están involucrados: Todos los que tienen algo para decir del producto (Stakeholders).
- El usuario dice lo que quiere cuando recibe lo que pidió. Es por esto, que las iteraciones cortas y las entregas continuas son esenciales, ya que permiten una retroalimentación y un enriquecimiento para futuras entregas y para una retro significativa
- No hay técnicas ni herramientas que sirvan para todos los casos. A veces se necesita trabajar con más prototipos o modelos, otras veces con técnicas sencillas es suficiente, pero hay que ver en cada caso que es lo que conviene.
- Lo importante no es entregar una salida, un requerimiento, sino es entregar un resultado, una solución de valor. Vamos a apostar a entregar una solución que le de valor al cliente, con esto nos referimos a algo que sea útil para el negocio o el cliente minimizando el desperdicio.

	TRADICIONAL	AGIL
Prioridad	Cumplir el plan	Entregar Valor
Enfoque	Ejecución ¿Cómo?	Estrategia (¿Por qué? ¿Para qué?)
Definición	Detallados y cerrados. Descubrimientos al inicio	Esbozados y evolutivos. Descubrimiento progresivo
Participación	Sponsors, stakeholder de mayor poder e interés.	Colaborativo con stakeholders de mayor interés (clientes, usuarios finales)
Equipo	Analista de negocios, Project Manager y Áreas de Proceso.	Equipo multidisciplinario.
Herramientas	Entrevistas, observación y formularios	Principalmente prototipado. Técnicas de facilitación para descubrir.
Documentación	Alto nivel de detalle. Matriz de Rastreabilidad para los requerimientos.	Historias de Usuario. Mapeo de historias (Story Mapping)
Productos	Definidos en alcance	Identificados progresivamente
Procesos	Estables, adversos al cambio.	Incertidumbre, abierto al cambio

User Stories: Concepto y las 3 "C"

Una **User Story (HU)** no es una especificación formal ni un documento contractual exhaustivo, sino una técnica de captura ligera que actúa como un **recordatorio o token para sostener una conversación** entre el equipo y el *Product Owner*

Propuesto por Ron Jeffries, una historia de usuario se compone de tres aspectos críticos (las 3 "C"):

1. **Card (Tarjeta):** La representación visible o física de la historia. Contiene la frase resumida y notas básicas que expresan la intención y el valor de negocio.
2. **Conversation (Conversación):** Es la **parte invisible y más importante** de la User Story. Es el diálogo continuo entre el equipo de desarrollo y el PO para descubrir e intercambiar los detalles, reglas e intenciones de la funcionalidad.
3. **Confirmation (Confirmación):** Constituye los criterios y pruebas de aceptación que verifican y confirman que la historia se ha programado e implementado correctamente conforme a las expectativas del cliente.

Formato de Redacción y Modelado de Roles

Estructura Estándar: Para mantener el foco en el valor, la tarjeta de una User Story se redacta bajo la plantilla

Como **Rol**, quiero **Actividad**, para **Valor**

- **Rol:** Quién realiza la acción o recibe el valor.
- **Actividad:** La función o acción que ejecutará el sistema.
- **Valor:** Justifica de forma explícita por qué es necesaria la funcionalidad para el negocio

Modelado de Roles: Se deben definir **roles específicos y concretos** de usuarios del sistema (ej. Conductor, Cajero), evitando utilizar el término genérico "usuario". Asimismo, se deben evitar los "usuarios representantes" o proxies (como gerentes, vendedores o capacitadores) en sustitución de los verdaderos usuarios finales

Naturaleza Multipropósito: La historia sirve simultáneamente como expresión de necesidad, descripción de producto, ítem de planificación y mecanismo para diferir conversaciones futuras

Criterios de Aceptación: Definen los **límites** de la historia y establecen los requerimientos funcionales mínimos requeridos para que el PO considere que la User Story aporta valor.

- **Buenas prácticas:** Deben definir una **intención y no una solución** (el qué, no el cómo), ser **independientes de la implementación** y mantenerse en un **alto nivel** sin saturarse de detalles de interfaz o código.
- Permiten a los desarrolladores saber cuándo parar de codificar y a los testers derivar los casos de prueba.

Pruebas de Aceptación: Se especifican al dorso de la tarjeta y expresan detalladamente los escenarios concretos de prueba con resultados esperados comprobables (anotando si el test **PASA** o **FALLA**).

El Modelo INVEST (Calidad de una User Story)

Creado por Bill Wake, el acrónimo **INVEST** establece las seis condiciones que debe cumplir una User Story para considerarse bien redactada y lista para su implementación.

- **I — Independent (Independiente):** Se puede calendarizar e implementar en cualquier orden sin depender fuertemente de otra historia.
- **N — Negotiable (Negociable):** Transmite la intención y el *qué*, dejando los detalles técnicos abiertos a la negociación con el equipo.
- **V — Valuable (Valiosa):** Aporta un valor de negocio claro y demostrable para el cliente o usuario final.
- **E — Estimable (Estimable):** El equipo comprende la historia lo suficiente como para calcular el esfuerzo necesario para su desarrollo.
- **S — Small (Pequeña):** Su tamaño está ajustado para que pueda ser consumida e implementada completamente dentro de un solo *Sprint*.
- **T — Testable (Testable / Comprobable):** Posee criterios de aceptación claros que permiten demostrar mediante pruebas si fue correctamente implementada.

Spikes (Historias de Investigación)

Un Spike es un tipo especial de historia de usuario orientada a reducir la incertidumbre y mitigar riesgos técnicos o funcionales críticos antes de abordar la implementación de una característica

- **Spike Técnico:** Investigación o prototipado para evaluar una nueva tecnología, arquitectura o tiempo de respuesta.
- **Spike Funcional:** Elaboración de un prototipo para evaluar la interacción o la experiencia con el usuario y obtener retroalimentación sobre la solución.
- **Lineamientos:** Los Spikes se consideran la excepción y no la regla; se aplican a incógnitas grandes y deben ejecutarse en una iteración previa a la historia de usuario correspondiente.

Estimaciones

la estimación es la **predicción del tamaño, esfuerzo, tiempo y costos de un proyecto en un contexto de alta incertidumbre. Estimar no es planificar ni comprometerse;** la estimación calcula la probabilidad de magnitud del trabajo, mientras que el plan y el compromiso establecen los objetivos de negocio negociables.

El Cono de Incertidumbre (Cone of Uncertainty)

Al inicio de un proyecto (fase de factibilidad o concepción), la incertidumbre es máxima; una estimación inicial puede variar drásticamente respecto al costo/tiempo real. A medida que el proyecto avanza, se obtiene más información y se ejecutan fases de diseño e implementación, la incertidumbre disminuye progresivamente hasta converger al entregar el producto.

Variables Clave Para Estimar

- **Tamaño:** Magnitud del software a construir (contar requerimientos, Casos de Uso, líneas de código o User Stories).
- **Esfuerzo:** Horas o meses-persona lineales (ideales) necesarios para desarrollar la solución.
- **Calendario (Duración):** Distribución del esfuerzo en días/meses calendario asignando personal.
- **Costo y Presupuesto:** Cálculo monetario derivado del esfuerzo y las tarifas de mano de obra.
- **Recursos Críticos:** Definición de equipamiento, licencias e integrantes requeridos.

Técnicas Tradicionales de Estimación

- **Juicio de Experto (Puro y Estructurado):** Basado en la experiencia de especialistas. Para estructurarlo se recomienda descomponer el problema en tareas de granularidad adecuada.
- **Wideband Delphi:** Método de consenso grupal e iterativo. Un coordinador entrega especificaciones a expertos que envían estimaciones **anónimas**. Tras revisar las desviaciones sin conocer las identidades de sus pares, los expertos vuelven a estimar en rondas sucesivas hasta que los números convergen.
- **Analogía / Datos Históricos:** Comparación sistemática del proyecto actual con proyectos similares desarrollados previamente por la organización.
- **Modelos Algorítmicos (ej. COCOMO / COCOMO II de Boehm):** Fórmulas matemáticas basadas en métricas cuantitativas como KSLOC (miles de líneas de código) o Puntos de Función ajustados por multiplicadores de esfuerzo (complejidad, experiencia, restricciones).
- **Actividades Omitidas (Errores Comunes):** Los desarrolladores tienden al optimismo. Suelen omitirse horas de testing, documentación técnica, reuniones, licencias y ausencias, generando desvíos en el calendario

Estimaciones Ágiles

Filosofía: Separar Tamaño de Duración

En proyectos tradicionales, se intenta estimar directamente el tiempo (horas/días) que tomará cada tarea, lo que introduce altos márgenes de error debido al sesgo individual. La estimación ágil **separa completamente la estimación del tamaño/complejidad de la estimación de la duración**.

1. El equipo estima únicamente el **tamaño relativo** del trabajo.
2. La **duración** se calcula empíricamente utilizando la **velocidad** del equipo.

Estimación Relativa

Las personas son deficientes estimando valores absolutos en tiempo, pero muy efectivas comparando elementos entre sí ("esta historia es el doble de compleja que aquella"). Se utiliza una **User Story Canónica** (un estándar de referencia del equipo) contra la cual se comparan los nuevos requerimientos.

La Unidad Ágil: Story Points (Puntos de Historia)

Es una unidad de medida abstracta y propia de un equipo específico que combina tres factores:

1. **Complejidad** del requerimiento.
2. **Esfuerzo** requerido.
3. **Incertidumbre / Riesgo** involucrado

Propiedades:

- **Relativo e Inconmensurable:** 1 SP de un equipo NO equivale a 1 SP de otro equipo.
- **No caduca:** Un valor asignado en Story Points no pierde validez cuando el equipo adquiere más experiencia tecnológica, a diferencia de los días ideales.
- **No es tiempo:** No representa horas hombre fijas.

Escala de Estimación: Serie de Fibonacci Modificada

Se utilizan escalas no lineales para reflejar que, a mayor tamaño de un requerimiento, la incertidumbre y complejidad crecen de forma exponencial.

Escala: 0, $\frac{1}{2}$, 1, 2, 3, 5, 8, 13, 20, 40, 100

Decodificación de valores:

- 0/ $\frac{1}{2}$ /1: Funcionalidad mínima o trivial.
- 2/3/5: Funcionalidades de tamaño pequeño a mediano (el tamaño óptimo para trabajar en un Sprint).
- 8: Funcionalidad grande. Es el límite máximo deseable para ingresar a un Sprint; si es mayor, conviene dividirla.
- 13/20: Demasiado grande para implementarse con certeza en una iteración; requiere subdivisión (*desagregación*).
- 40/100: Épicas o requerimientos con desconocimiento crítico; no deben iniciarse sin antes investigar (*Spike*) o replantear.

Planning Poker (Poker de Planificación)

Técnica colaborativa popularizada por Mike Cohn que combina juicio de expertos, analogía y desagregación.

- **Preparación:** Cada desarrollador recibe un mazo de cartas con la secuencia de Fibonacci.
- **Presentación:** El *Product Owner* (o moderador) lee la User Story y aclara dudas.
- **Votación Oculta:** Cada desarrollador selecciona una carta y la coloca boca abajo para evitar el **sesgo de anclaje**.
- **Revelación Simultánea:** Todas las cartas se muestran al mismo tiempo.
- **Discusión de Extremos:** Si hay divergencia, los integrantes con la estimación más alta y más baja justifican sus razones técnicas o supuestos ocultos.
- **Re-votación:** Se repite el ciclo hasta alcanzar un consenso grupal sobre el valor final.

Métrica de Progreso y Duración: Velocidad (Velocity)

Definición: Cantidad de Story Points correspondientes a Historias de Usuario completadas al **100%** (*Definition of Done*) durante una iteración.

Regla de Oro: No se otorgan créditos parciales a historias incompletas al finalizar el Sprint.

La Ecuación Central Ágil:

Duración del proyecto (sprints) = suma de SP del backlog / velocidad promedio del equipo (puntos/sprint)

Autocorrección Empírica: La velocidad absorbe de forma automática las fluctuaciones o errores de estimación sin necesidad de estar re-estimando las tareas constantemente, salvo que el alcance o tamaño relativo de una historia sufra un cambio estructural explícito.

Scrum

Scrum: El Framework

Scrum es un marco de trabajo liviano diseñado para ayudar a las personas, equipos y organizaciones a generar valor mediante soluciones adaptativas frente a problemas complejos. Es un marco intencionalmente incompleto que no prescribe instrucciones ni procesos detallados, sino que establece reglas fundamentales para guiar las relaciones humanas, la interacción y la toma de decisiones. Scrum abarca el ciclo de vida completo del producto —desde el descubrimiento de necesidades hasta el desarrollo, la entrega, la operación y el retiro—, permitiendo gestionar el riesgo de forma proactiva y obtener aprendizaje validado en entornos de alta incertidumbre.

Los Principios de Scrum

Los principios de Scrum guían la dinámica de trabajo y la estructura del marco:

- **Control de proceso empírico:** El conocimiento proviene de la experiencia directa y la toma de decisiones se fundamenta en la evidencia observada.
- **Auto-organización / Autogestión:** El equipo decide internamente *quién, qué, cómo, cuándo y dónde* trabaja, eliminando la dirección jerárquica o el microcontrol externo.
- **Colaboración:** Trabajo conjunto e interactivo continuo entre los miembros del equipo y los interesados (*stakeholders*).
- **Priorización basada en valor:** Ordenamiento constante del trabajo enfocado en maximizar el retorno e impacto de negocio a largo plazo.
- **Time-boxing (Límites de tiempo):** Uso de bloques temporales fijos para los eventos, lo que reduce la complejidad cognitiva y establece un ritmo de trabajo constante (*cadencia*).
- **Desarrollo iterativo e incremental:** Avance mediante ciclos cortos que reducen el costo del error y permiten ajustar el rumbo rápidamente.

El Motor: Empirismo y el Ciclo OODA

El motor de Scrum integra los tres pilares tradicionales del empirismo con la dinámica de respuesta rápida del **Ciclo OODA** (Observar, Orientar, Decidir, Actuar), popularizado por John Boyd para decisiones dinámicas:

Los 3 Pilares del Empirismo:

- **Transparencia:** Muestra la realidad tal como es. El proceso, los objetivos y el trabajo emergente deben ser totalmente visibles para quienes realizan y reciben el trabajo.
- **Inspección:** Evaluación frecuente y diligente de los artefactos y del avance hacia las metas para detectar desviaciones o problemas.
- **Adaptación:** Ajuste inmediato del proceso o producto cuando se detecta un desvío. La inspección sin adaptación se considera un desperdicio.

La Lente OODA en Scrum:

- **Observar (Observe):** Recopilar evidencia objetiva y perspectivas diversas apoyándose en la Apertura y el Respeto.
- **Orientar (Orient):** Usar el Coraje para interpretar la realidad, desafiar suposiciones y ajustar la dirección en entornos inciertos.
- **Decidir (Decide):** Mantener el Foco para seleccionar e implementar experimentos paralelos seguros para fallar (*safe-to-fail probes*).
- **Actuar (Act):** Compromiso de ejecución para convertir las decisiones en incrementos utilizables dentro de las restricciones de tiempo.

Valores de Scrum

El éxito de Scrum depende de que las personas incorporen y vivan cinco valores esenciales:

- **Compromiso (Commitment):** Dedicación compartida a alcanzar las metas del Sprint y del Producto, apoyándose mutuamente como equipo.
- **Foco (Focus):** Concentración prioritaria en el trabajo del Sprint actual para lograr el mayor progreso hacia el Objetivo del Sprint.
- **Apertura / Franqueza (Openness):** Transparencia del equipo e interesados sobre el estado real del trabajo, los aprendizajes y los desafíos.
- **Respeto (Respect):** Valoración mutua como profesionales capaces e independientes, promoviendo la diversidad de opiniones.
- **Coraje (Courage):** Valentía para hacer lo correcto, abordar problemas complejos, decir "no" cuando corresponde y asumir riesgos controlados.

Valores y Cultura: El Sistema Operativo de la Confianza

Los valores de Scrum constituyen el "**Sistema Operativo de la Confianza**" sobre el cual se sostiene la cultura organizacional. En entornos complejos, la confianza es la moneda de cambio fundamental; sin Coraje para decir "no" o señalar desviaciones, y sin Respeto para criticar las ideas y no a las personas, el control empírico se desmorona.

La cultura de confianza permite articular el Ciclo OODA de forma fluida: la **Apertura y el Respeto** garantizan que al **Observar** no se oculten datos; el **Coraje** permite **Orientar** e interpretar la realidad sin sesgos; el **Foco** permite **Decidir** el camino óptimo sin distracciones; y el **Compromiso** asegura **Actuar** con rigor técnico y responsabilidad.

La Unidad Central: Una Célula Profesional

El **Scrum Team** es la unidad fundamental de Scrum, concebida como una **célula profesional autogestionada**, pequeña (por lo general de 10 personas o menos) y sin subequipos ni jerarquías internas.

- **Autogestión:** El equipo decide internamente quién hace qué, cómo, cuándo y dónde, ejerciendo autonomía alineada hacia el Objetivo del Producto.
- **Profesionalismo y Responsabilidad Total:** Asume la responsabilidad integral de la solución "*desde la cuna hasta la tumba*", abarcando investigación, diseño, construcción, calidad, mantenimiento y operación.
- **Diversidad Cognitiva:** Polinización cruzada de habilidades técnicas y de negocio que permite abordar problemas complejos de forma integral.

Roles en la Guía Expandida

En la **Guía Expandida de Scrum (SGEP)**, la estructura de roles se extiende para diferenciar entre **roles generales** y **responsabilidades formales (accountabilities)** dentro del ecosistema del producto.

A diferencia del marco básico de 2020, el modelo ampliado define **4 roles en total** (Product Owner, Product Developer, Scrum Master y Stakeholder), de los cuales **solo 3 constituyen accountabilities dentro del Scrum Team**. Además, se formaliza la figura del **Supporter** como un tipo clave de Stakeholder.

Responsabilidades Formales (Accountabilities) del Scrum Team

El Scrum Team está integrado exclusivamente por tres *accountabilities*. Dentro del equipo no existen subequipos ni jerarquías, y se busca la diversidad cognitiva y la autogestión:

- **Product Owner (Obligatoriamente Humano):**
 - **Enfoque:** El "*Qué*" y el "*Por qué*" estratégico a largo plazo.
 - **Responsabilidades:** Es el líder del producto encargado de maximizar el valor. Define y comunica el **Product Goal**, gestiona y ordena el **Product Backlog**, y lidera la estrategia de negocio y la narrativa del producto.
 - **Compromiso de Valor:** Posee la palabra final sobre la **Definition of Outcome Done** (validación del impacto e hipótesis de valor en el mercado).
 - **Restricción:** Debe ser una única persona (no un comité ni un *proxy*) y ser obligatoriamente un ser humano.

- **Product Developer (Humano o Automatizado / IA):**
 - **Enfoque:** El "Cómo" técnico, la ejecución y la calidad.
 - **Composición e IA:** Puede estar conformado por desarrolladores humanos y por **agentes o herramientas de Inteligencia Artificial**. Bajo la regla del "Human in the loop", la responsabilidad final (accountability) de los resultados recae siempre al 100% en los humanos.
 - **Responsabilidades:** Crean el plan táctico (**Sprint Backlog**), adaptan el trabajo diariamente hacia el **Sprint Goal** e inculcan calidad.
 - **Compromiso Técnico:** Se adhieren estrictamente a la **Definition of Output Done** (calidad técnica e incremento utilizable). Se rigen bajo la premisa "Stop starting, start finishing" para optimizar el flujo.

- **Scrum Master (Obligatoriamente Humano):**
 - **Enfoque:** La efectividad del equipo y la transformación del entorno.
 - **Responsabilidades:** Actúa como líder servicial y **agente de cambio (Change Agent)** en todos los niveles organizacionales. Guía la adopción del empirismo, ayuda al equipo a autogestionarse y remueve los impedimentos organizacionales que el Scrum Team no puede resolver por sí mismo.
 - **Restricción:** Debe ser obligatoriamente un ser humano. No es un secretario, un administrador de herramientas ni un jefe de proyecto.

El Ecosistema Extendido de Roles

- **Stakeholder (Rol):**
 - Entidades, individuos o grupos interesados o afectados directamente por las actividades y resultados del producto (clientes, usuarios finales, decisores/choosers, sponsors financieros, legisladores o incluso entidades no humanas como la ley o la IA). Proveen retroalimentación, necesidades y validan el valor real entregado.

- **Supporter (Rol - Stakeholder Específico y Agente de Cambio):**
 - Es una adición fundamental de la Guía Expandida. Corresponde a una coalición de líderes, gerentes y ejecutivos que apoyan activamente la adopción de Scrum.
 - Función: Eliminan barreras sistémicas, modifican procesos organizacionales rígidos (HR, Finanzas, Compras), proveen recursos y brindan "cobertura aérea" para proteger la autogestión del Scrum Team.
 - Regla de diferenciación: Si un gerente o líder no empodera al equipo ni remueve impedimentos organizacionales, no califica como Supporter.

La Autogestión y la Inteligencia Artificial

Preservación de la Autonomía: El Scrum Team sigue siendo una célula profesional autogestionada que decide internamente *quién, qué, cómo, cuándo y dónde* se realiza el trabajo, sin dirección jerárquica ni microgestión externa.

Potenciación Cognitiva: La IA no ingresa para reemplazar la toma de decisiones del equipo, sino como un asistente o socio (*team mate*) que amplifica la capacidad de investigación, análisis de datos y exploración del equipo en entornos de alta incertidumbre.

Dentro de las accountabilities de Scrum, la función de Product Developer se define como la capacidad de crear incrementos utilizables. En la Guía Expandida, un Product Developer puede ser humano o automatizado (IA).

Funciones Principales de la IA en el Equipo:

- **Generación y automatización:** Apoyo en refactorización de código, generación de pruebas automatizadas y prototipado rápido.
- **Refinamiento y detección de patrones:** Análisis continuo del Product Backlog para descubrir ambigüedades, dependencias ocultas o inconsistencias.
- **Cuestionamiento de supuestos:** Actúa como una herramienta para poner a prueba hipótesis y desafiar las ideas del equipo (challenge thinking) antes de llevarlas a producción.

Regla de Oro (Accountability 100% Humana): La responsabilidad final y la rendición de cuentas (accountability) sobre la calidad técnica (Definition of Output Done) y el valor de negocio (Definition of Outcome Done) recaen siempre y en un 100% en las personas humanas del Scrum Team.

Uso Supervisado (Human-in-the-loop): La IA sugiere, analiza o borronea, pero el ser humano evalúa las evidencias, aplica el juicio ético y toma la decisión final de incorporar el trabajo al incremento.

Eventos de Scrum

El Sprint (Evento Contenedor)

- **Propósito:** Es el corazón de Scrum y actúa como el contenedor donde ocurren todos los demás eventos. Durante este periodo, las ideas se transforman en incrementos utilizables y potencialmente valiosos.
- **Duración (Timebox):** Duración fija de **1 mes o menos** para mantener la consistencia y reducir el riesgo. Un nuevo Sprint comienza inmediatamente después de finalizar el anterior.

Sprint Planning (Planificación del Sprint)

- **Propósito:** Inicia el Sprint definiendo el trabajo que se va a realizar mediante la colaboración de todo el equipo. Aborda tres aspectos críticos:
 1. **¿Por qué es valioso este Sprint?** (Define el **Sprint Goal**).
 2. **¿Qué se puede hacer?** (Selecciona ítems del Product Backlog).
 3. **¿Cómo se realizará el trabajo?** (Plan técnico de ejecución que conforma el Sprint Backlog).
- **Duración (Timebox):** Máximo **8 horas** para un Sprint de un mes (menor duración en Sprints más cortos).

Daily Scrum (Sincronización Diaria)

- **Propósito:** Inspeccionar el progreso diario hacia el **Sprint Goal** y adaptar el Sprint Backlog según sea necesario, ajustando el plan de las siguientes 24 horas. Es un evento diseñado por y para los Developers.
- **Duración (Timebox):** Evento de **15 minutos** que se realiza todos los días hábiles a la misma hora y en el mismo lugar para reducir la complejidad.

Sprint Review (Revisión del Sprint)

- **Propósito:** Es una sesión de trabajo interactiva y colaborativa entre el Scrum Team y los stakeholders. Se inspecciona el resultado del Sprint (el Incremento), se evalúa el progreso hacia el **Product Goal** y se ajusta el Product Backlog en función de los aprendizajes y cambios del entorno.
- **Duración (Timebox):** Máximo **4 horas** para un Sprint de un mes.

Sprint Retrospective (Retrospectiva del Sprint)

- **Propósito:** Concluye el Sprint. El Scrum Team reflexiona sobre cómo fue la iteración respecto a personas, interacciones, procesos, herramientas y su Definition of Done. Identifica las mejoras más impactantes para aumentar su calidad y efectividad, e implementa acciones concretas para el próximo Sprint.
- **Duración (Timebox):** Máximo **3 horas** para un Sprint de un mes.

Diferencias entre Output y Outcome (La Doble Definición de "Hecho")

Resumen Conceptual

- **Output (Salida / Entregable):** Es la construcción física o digital producida por el equipo (código, funcionalidades, documentación, tareas completadas).
- **Outcome (Resultado / Impacto):** Es el valor real de negocio y el cambio observable en el comportamiento del usuario que genera dicha salida.

"El Output es el requisito previo necesario; el Outcome es el éxito real del producto".

Definition of Output Done (Calidad Técnica):

- Es la descripción formal del estado del Incremento cuando cumple con todas las medidas de calidad técnica requeridas.
- Asegura la mantenibilidad, ausencia de deuda técnica y la debida diligencia en el código.
- **Innegociable:** El estándar de calidad técnica nunca se degrada por presión de tiempo o compromisos de fecha.

Definition of Outcome Done (Validación de Valor):

- Es el compromiso que describe las evidencias observables (cuantitativas o cualitativas) requeridas para demostrar beneficios reales alcanzados.
- Se mide mediante telemetría del producto, analíticas de comportamiento o retroalimentación directa del usuario.
- **El principio del empirismo:** El valor de una funcionalidad sigue siendo una **mera suposición o hipótesis** hasta que se valida con evidencia real de mercado.

Artefactos

Product Backlog (Lista del Producto)

- **Definición:** Es una lista emergente, dinámica y ordenada de todo lo que se requiere para mejorar el producto. Representa la **única fuente de trabajo** a realizar por el Scrum Team.
- **Compromiso asociado: Objetivo del Producto (Product Goal).** Describe una meta u objetivo estratégico a mediano/largo plazo que sirve como brújula para que el equipo planifique y mantenga el enfoque.

Sprint Backlog (Lista del Sprint)

- **Definición:** Es la imagen visible en tiempo real del plan táctico realizado **por y para los Developers**. Se compone de tres partes:
 1. **El Por qué:** El Objetivo del Sprint.
 2. **El Qué:** El conjunto de elementos seleccionados del Product Backlog.
 3. **El Cómo:** Un plan de acción detallado para entregar el Incremento.
- **Compromiso asociado: Objetivo del Sprint (Sprint Goal).** Es el propósito único y unificador de la iteración que otorga flexibilidad en cuanto al trabajo exacto a ejecutar para lograrlo.

Incremento (Increment)

- **Definición:** Es un peldaño o paso concreto hacia el Objetivo del Producto. Se conforma por la suma de todos los elementos del Product Backlog completados durante el Sprint actual, perfectamente integrados con los incrementos de Sprints anteriores. Para aportar valor, debe ser **totalmente utilizable**.
- **Compromiso asociado:**
 - **Definición de Terminado (Definition of Done):** Es la descripción formal del estado del Incremento cuando cumple con las medidas y estándares de calidad exigidos. Si un ítem no cumple con esta definición, no puede presentarse ni liberarse.
 - **Evolución en la Guía Expandida (SGEP):** Se formaliza una doble definición dividiendo el compromiso técnico en **Definition of Output Done** (calidad del Incremento y ausencia de deuda técnica) y la validación del valor en **Definition of Outcome Done** (evidencia observable del impacto e hipótesis de negocio cumplidas en el Producto).

Pensamiento Orientado a Producto (Product Thinking vs. Project Thinking)

De proyectos a productos: Los proyectos son temporales, tienen un fin fijo y a menudo dejan un software "huérfano" tras su entrega. Los productos son activos continuos que deben cuidarse "desde la cuna hasta la tumba" (cradle to grave), abarcando creación, mantenimiento, operación y retiro.

Integración de Discovery y Delivery : La investigación y descubrimiento de necesidades (Product Discovery) no es una fase previa separada, sino una actividad continua que **ocurre dentro del mismo Sprint** junto con la entrega técnica (Delivery).

Visión del Producto (Product Vision)

- **El Norte Verdadero (True North):** Es la aspiración inspiradora a largo plazo que describe el estado futuro deseado del producto.
- **Hipótesis amplia:** La guía enfatiza que una visión suele ser una hipótesis a largo plazo o "una obra de ficción" hasta que se empieza a validar en el mercado. Sirve como marco de referencia, pero debe mantenerse abierta a adaptarse si la evidencia demuestra lo contrario.

Objetivo del Producto (Product Goal)

- **El compromiso del Product Backlog:** Es el objetivo estratégico a **mediano plazo** formalmente asociado al *Product Backlog*.
- **Corte Vertical Tangible:** Para evitar que la Visión del Producto resulte abrumadora o inalcanzable, el *Product Goal* actúa como un "**corte vertical**" tangible y medible que acerca al equipo paso a paso a esa visión.
- **Enfoque secuencial:** El *Scrum Team* debe cumplir o abandonar un *Product Goal* antes de asumir el siguiente, garantizando el foco y evitando la dispersión en múltiples iniciativas paralelas.

Refinamiento del Product Backlog (Refinement)

- **Organismo vivo:** El Product Backlog no es un plan estático de requerimientos, sino un flujo emergente donde *"la claridad precede a la velocidad"*.
- **Proceso emergente y continuo:** El refinamiento es una actividad permanente para descomponer ítems grandes o difusos en unidades más pequeñas, comprensibles y listas para ser implementadas dentro de un Sprint (Ready).
- **Reducción de riesgo:** Se utiliza para investigar, realizar spikes, validar supuestos y reducir la incertidumbre técnica y de negocio.
- **Criterios de Salida y Resultado (Acceptance & Outcome Criteria):** Durante el refinamiento, los elementos del backlog no solo incorporan criterios de aceptación técnica (el Qué), sino también **criterios de resultado (Outcome Criteria - el Por qué)** para guiarse por el impacto real en el usuario y no por la simple acumulación de código.

Capacidad del Equipo en un Sprint

La capacidad total del Sprint no equivale simplemente a multiplicar la jornada laboral teórica por el número de desarrolladores. Para determinar la capacidad real disponible, se descuentan las distintas actividades e interrupciones del equipo:

- **Capacidad de trabajo en los PBIs:** El tiempo neto efectivo dedicado al desarrollo de las historias de usuario y tareas seleccionadas.
- **Tiempo fuera del personal:** Ausencias programadas, vacaciones, licencias, feriados o días personales de los miembros del equipo.
- **Actividades del propio Sprint:** El tiempo invertido en los eventos de Scrum y actividades continuas (Sprint Planning, Sprint Review, Sprint Retrospective y Refinamiento/Grooming).
- **Compromisos externos:** Asignaciones a tareas fuera del Sprint, tales como soporte operativo, mantenimiento de sistemas legados o dedicación parcial a otros proyectos.
- **Buffer / Holgura del Sprint (Slack):** Margen de tiempo reservado para gestionar la incertidumbre, imprevistos o interrupciones, evitando que el equipo se sobrecargue y comprometa la calidad.

Capacidad vs. Velocidad

- **Capacidad (Capacity):** Es una métrica **de entrada (input)** expresada en horas de esfuerzo disponibles para el Sprint actual, influenciada por la disponibilidad cambiante de las personas (licencias, reuniones, feriados).
- **Velocidad (Velocity):** Es una métrica **de salida (output)** expresada en Story Points que mide el volumen histórico de trabajo completado al 100% (Definition of Done) en Sprints pasados.

Planificación de Release y Sprint

Los Niveles de Planificación (La Cebolla de Planificación)

La agilidad aborda la planificación como un proceso continuo que se despliega en múltiples niveles concéntricos:

1. **Estrategia / Portafolio:** Alineación con los objetivos de alto nivel de la organización.
2. **Producto:** Definición de la Visión y el Product Roadmap.
3. **Release:** Planificación a mediano plazo (meses/semanas) orientada a la liberación de valor comercial.
4. **Iteración / Sprint:** Planificación a corto plazo (semanas: 1 a 4 semanas) para construir incrementos utilizables.
5. **Día (Daily Scrum):** Sincronización y ajuste táctico diario cada 24 horas.

Planificación de Release (Release Planning)

Es el proceso que conecta la visión del producto con la ejecución de los Sprints, determinando cómo y cuándo se entregarán incrementos significativos de valor (releases) a los usuarios o al mercado.

- **El Triángulo de Restricciones en la Release:** A diferencia del enfoque tradicional, se gestionan el **Alcance (Scope)**, la **Fecha (Date)** y el **Presupuesto (Budget)**. Frecuentemente, la fecha y el presupuesto se mantienen fijos, mientras que el alcance se gestiona de forma flexible para maximizar el valor entregado.
- **Entradas (Inputs):**
 - Visión del Producto y Product Roadmap.
 - Product Backlog ordenado por valor y estimado en Puntos de Historia.
 - **Velocidad histórica del equipo:** La métrica empírica de Story Points completados por Sprint.
- **Salidas (Outputs):**
 - Mapa de Sprints (Sprint map).
 - Conjunto de características mínimas comercializables (MRFs / MMFs).
 - Rango proyectado de entregas (fechas/costos o alcance).

Cadencia de Releases: Un release no tiene por qué ocurrir solo al final del proyecto; puede realizarse tras **múltiples Sprints**, al finalizar **cada Sprint**, o incluso tras **cada funcionalidad (Continuous Delivery)**.

Planificación del Sprint (Sprint Planning)

Es el primer evento formal de cada Sprint, donde todo el *Scrum Team* colabora para diseñar el plan de trabajo táctico de la iteración.

- **Duración (Timebox):** Un máximo de **8 horas** para un Sprint de un mes (proporcionalmente menor para Sprints de menor duración).
- **Los 3 Temas Fundamentales de la Sprint Planning:**
 1. **Tema 1 — ¿Por qué es valioso este Sprint? (The Why):** El Product Owner propone cómo el producto incrementará su valor y todo el equipo define de forma colaborativa el **Sprint Goal**.
 2. **Tema 2 — ¿Qué se puede hacer en este Sprint? (The What):** Los Developers seleccionan elementos del Product Backlog que cumplan con el criterio de preparación (Ready). La cantidad seleccionada se pronostica considerando su capacidad actual, rendimiento pasado y la Definition of Output Done.
 3. **Tema 3 — ¿Cómo se realizará el trabajo elegido? (The How):** Los Developers diseñan el plan de ejecución técnico descomponiendo las Historias de Usuario en tareas concretas (por lo general de un día o menos). Ninguna persona externa le indica a los Developers cómo transformar los PBIs en un Incremento.
- **Artefacto Resultante:** El **Sprint Backlog**, compuesto por el Objetivo del Sprint (Por qué), los PBIs seleccionados (Qué) y el plan técnico de tareas (Cómo).
- **Buffer / Holgura (Slack):** Los equipos maduros planifican margen o holgura para gestionar la incertidumbre e imprevistos sin sobrecargarse ni comprometer la calidad o el Sprint Goal.

Métricas y Herramientas

La Filosofía Ágil sobre las Métricas

- **La Regla de Oro:** "La medición es una salida, no una actividad". Se aplica una filosofía minimalista: **medir solo lo que sea necesario y nada más**.
- **Medida Principal de Progreso:** Guiado por el Manifiesto Ágil, la métrica reina es el **Software Trabajando (Running Tested Features - RTF)**.
- **El Empirismo por encima de la Predicción:** Los gráficos y analíticas ayudan a proyectar, pero en entornos complejos jamás reemplazan el aprendizaje basado en lo que ya ha sucedido.

Gráficos de Seguimiento y Radiadores de Información:

Sprint Burndown Chart (Gráfico de Trabajo Pendiente del Sprint)

- Muestra la cantidad de trabajo restante (en horas o Story Points) a lo largo de los días del Sprint.
- Compara la **línea de avance real** contra una **línea ideal** proyectada para identificar si el equipo se encuentra a tiempo, adelantado o atrasado respecto al Sprint Goal.

Release Burnup Chart (Gráfico de Trabajo Completado del Release)

- Grafica el **acumulado de trabajo terminado (DONE)** Sprint a Sprint hacia el objetivo final (Scope).
- Su principal ventaja frente al Burndown es que **hace visibles los cambios en el alcance (scope creep)**, mostrando claramente si una desviación se debe a un retraso del equipo o a la adición de nuevos requerimientos.

Diagrama de Flujo Acumulativo (CFD - Cumulative Flow Diagram)

- Muestra la evolución del estado de los ítems a lo largo de las distintas etapas del proceso (To Do, In Process, To Verify, Done).
- Es la herramienta ideal para monitorear el **Trabajo en Progreso (WIP)** y detectar cuellos de botella en el flujo de entrega.

Gestión del Software como producto

SCM

Gestión de Configuración de Software (SCM - Software Configuration Management) es una disciplina de soporte de la ingeniería de software descrita como una actividad "paraguas", ya que es transversal a todo el proyecto y acompaña al producto a lo largo de todo su ciclo de vida.

Su propósito fundamental es establecer y mantener la integridad de los productos de software mediante la identificación de sus componentes, el control sistemático de sus cambios, la verificación de su correspondencia con los requerimientos y el registro de su evolución.

SCM existe para evitar problemas críticos en la gestión de componentes, tales como la pérdida de artefactos, la superposición de cambios, el doble mantenimiento, la regresión de fallas o la existencia de cambios no validados.

Garantizar la **integridad del producto** a través de SCM significa asegurar que el software:

- **Satisface las necesidades del usuario** (hace lo que se espera que haga).
- **Permite la trazabilidad completa:** Existen vínculos entre los componentes que permiten analizar con precisión qué partes se verán afectadas ante un cambio de requerimientos.
- **Cumple con los criterios de desempeño y requerimientos no funcionales.**
- **Satisface las expectativas de costo y presupuesto.**

Conceptos Fundamentales de SCM

Ítem de Configuración de Software (SCI / IC): Es todo artefacto del proyecto o producto que se almacena en un repositorio, puede sufrir cambios a lo largo del tiempo y sobre el cual es necesario conocer su estado y evolución. Se clasifican por su ciclo de vida en:

- Ítems de Producto: Duran mientras el producto exista (ej. arquitectura, ERS, código fuente, casos de prueba, manuales).
- Ítems de Proyecto: Duran lo que dura el proyecto (ej. plan de proyecto, cronograma).
- Ítems de Iteración: Duran lo que dura el sprint (ej. plan de iteración, reporte de defectos, burn-down chart).

Repositorio: Es el contenedor estructurado que almacena los ítems de configuración, guarda el historial de sus versiones, sus atributos y relaciones, y permite evaluar el impacto de los cambios. Puede ser centralizado (un único servidor central) o descentralizado (cada desarrollador tiene una copia completa del repositorio).

Versión: Instancia o estado particular de un ítem de configuración en un instante dado.

Variante: Versión de un ítem de configuración que evoluciona por separado para representar configuraciones alternativas (por ejemplo, para adaptar el software a distintos sistemas operativos o plataformas).

Configuración del Software: Es la "foto" o suma de todos los ítems de configuración con sus versiones específicas en un momento determinado del tiempo.

Línea Base (Baseline): Es una configuración de software que ha sido revisada y acordada formalmente, sirviendo como punto de referencia estable para desarrollos posteriores. Una vez establecida, **solo puede modificarse mediante un procedimiento formal de control de cambios**. Se dividen en:

- Líneas Base de Especificación: Contienen documentos formales (requerimientos, diseño).
- Líneas Base Operacionales: Contienen productos ejecutables que han superado pruebas de calidad (la primera línea base operacional coincide con la primera Release).

Rama (Branch): Bifurcación del desarrollo que permite experimentar o resolver fallas en paralelo sin desestabilizar la rama principal (trunk/master). Su reintegración se realiza mediante la operación de **Merge**.

Las 4 Actividades y Elementos Principales de SCM

1 Identificación de Ítems de Configuración

Consiste en identificar unívocamente cada ítem, definir la estructura del repositorio, establecer las reglas y convenciones de nombrado/versionado y seleccionar qué componentes formarán parte de cada línea base.

2 Control de Cambios

Es el procedimiento formal necesario para modificar cualquier ítem que ya pertenece a una **línea base**.

- **Comité de Control de Cambios (CCB):** Grupo multidisciplinario (analistas, desarrolladores, testers, cliente) encargado de evaluar y autorizar los cambios.
- **Flujo del proceso:** Se recibe una propuesta de cambio, El CCB realiza un **análisis de impacto** (evaluando costo, esfuerzo técnico, efectos colaterales y riesgos). El CCB aprueba o rechaza la solicitud, se emite una orden de cambio, se ejecuta la modificación y se realiza una revisión de partes. Por último se notifica a los interesados y se establece la nueva línea base.

3 Auditorías de Configuración de Software

Son evaluaciones independientes y objetivas para comprobar que el producto desarrollado coincide con lo especificado en las líneas base:

- **Auditoría Física de Configuración (PCA):** Asegura que todos los ítems declarados en la línea base estén realmente contruidos y presentes en la entrega.
- **Auditoría Funcional de Configuración (FCA):** Evalúa de forma independiente que la funcionalidad y el rendimiento reales coincidan con la especificación de requerimientos.
- Sirven de soporte para la **Validación** (verificar que se resuelve el problema correcto del cliente) y la **Verificación** (demostrar que el producto cumple con los objetivos preestablecidos en las líneas base).

4 Informes y Registro de Estado (Status Accounting)

Es el registro administrativo que mantiene el historial de la evolución del sistema. Responde preguntas clave como: ¿cuál es el estado de un ítem?, ¿se aprobó una solicitud de cambio?, ¿qué versión implementa una mejora específica? o ¿cuáles son las diferencias exactas entre dos versiones?

Planificación de SCM

Todas las actividades de SCM deben documentarse al inicio del proyecto en el **Plan de Gestión de Configuración de Software**. Este documento define la estructura del repositorio, los integrantes del CCB, las herramientas de automatización, los estándares de nombrado y los procedimientos de auditoría y cambio.

Evolución de SCM en Ambientes Ágiles y DevOps

En marcos ágiles y culturas de entrega continua, SCM cambia de enfoque:

- **De la burocracia a la automatización:** SCM pasa de ser un mecanismo de control prescriptivo sobre los desarrolladores a ser un servicio que **facilita la coordinación y elimina el desperdicio**.
- **Eliminación de comités rígidos:** Las auditorías externas tradicionales y los comités formales (CCB) se relativizan o eliminan para no frenar la autogestión.
- **Prácticas de CI/CD:**
 - **Integración Continua (CI):** Integración frecuente del código en un repositorio compartido mediante compilaciones y pruebas unitarias automatizadas.
 - **Entrega Continua (Continuous Delivery):** Automatización de pruebas de aceptación para asegurar que el software esté siempre en un estado de "listo para producción", requiriendo solo la decisión del Product Owner para liberarse.
 - **Despliegue Continuo (Continuous Deployment):** Despliegue automatizado directo al entorno del usuario final a través de pipelines sin intervención humana manual.